Computing (2024) 106:2361–2387
https://doi.org/10.1007/s00607-024-01289-8

REGULAR PAPER

Semantically realizing discovery and composition for RESTful web services

Haijun Gu[1] · Yingyu Ma[1] · Siqi Wang[1] · Xincheng Chen[1] · Weihua Su[1]

Received: 27 May 2023 / Accepted: 2 April 2024 / Published online: 23 April 2024
© The Author(s), under exclusive licence to Springer-Verlag GmbH Austria, part of Springer Nature 2024

Abstract

The processes of service discovery and composition are crucial tasks in application development driven by Web Services. However, with RESTful Web Service replacing SOAP-based Web Service as the dominant service-providing approach, the research on service discovery and composition should also shift its focus from SOAP-based Web Service to RESTful Web Service. The unstructured, resource-oriented and unified interface characteristics of RESTful Web Service pose challenges to its discovery and composition process. In this work, a framework for implementing RESTful Web Service discovery and automatic composition based on semantic technology is proposed. Firstly, the framework uses the OpenAPI Specification (OAS), which is extended by resource attributes, as the RESTful Web Service description specification, and then supports semantic-based matching discovery and automatic composition by attaching the concepts of domain ontology to the extended OAS. Secondly, the framework is fully adapted to REST features and provides a method for building service composition dependencies during registration, which is used to generate composition schemes during the service discovery process. Finally, the framework provides a discovery method that can return RESTful Web services to the requester in the form of single-point services or service composition schemes according to the magnitude of the semantic similarity with the requester's requirements. We applied the proposed methods to experiment with RESTful Web services in three different fields, and the results show that the methods effectively calculate the similarity between RESTful single-point Web services or composite Web services and service requests with the support of domain ontology.

Keywords RESTful web service · RESTful web service description · RESTful web service discovery · RESTful web service composition · Semantics

Mathematics Subject Classification 68U35 · 68T35 · 68Q55 · 68T30

Extended author information available on the last page of the article

1 Introduction

Service-oriented computing effectively facilitates application development by consuming and reusing Web Services [1, 2]. Web Service is a platform-independent, loosely coupled, self-contained, and programmable web application that provides its functionality to external calls by exposing APIs on the Web [3]. In the basic Web Service architecture of "Service Provider-Registration Center-Service Requester", service providers submit their developed services to the registration center. Service requesters (application developers, not end users) discover suitable services in the registration center based on their own program development needs. By consuming and reusing discovered Web services, the application development cycle of the service requester will be significantly shortened [4]. At the same time, when a single service in the registration center cannot meet the requester's needs, it can be considered to combine several services to jointly achieve the functions required by the requester [5]. From the above process, it is not difficult to see that service discovery and composition are key links in the application development driven by Web Services, and the study of both is of great significance.

Early Web Service standards consisted of three main parts: SOAP (Simple Object Access Protocol), WSDL (Web Services Description Language), and UDDI (Universal Description, Discovery, and Integration) [6]. With the continuous progress of internet and Web technologies, the heavy SOAP protocol based on XML and complex WSDL documents are increasingly criticized. Meanwhile, under the advocacy of large internet companies such as Yahoo, Google, and Facebook, REST(Representative State Transfer) has been widely accepted in the Web field as a simpler alternative to SOAP-based Web Service. Service providers such as Google have abandoned or moved away from SOAP-based interfaces in favor of an easier-to-use, resource-oriented model for exposing their services, known as RESTful Web Services. However, as far as we know, although RESTful Web Service has replaced SOAP-based Web Service as the dominant service-providing approach, the focus of the research on Web Service discovery and composition is still on SOAP-based Web Service. Relatively little research has been done on RESTful Web Service discovery and composition [7].

REST is a software architecture style proposed by American computer scientist Roy Fielding in his doctoral thesis in 2000. It serves as a design and development pattern for Web applications, aiming to reduce the complexity of development and improve the scalability of the system. We generally refer to Web services following the REST architecture style as RESTful Web Services. To be considered RESTful, a Web service should adhere to several constraints. Firstly, it should be resource-oriented, with resources having multiple representations and uniquely identified through URIs. JSON is recommended as the preferred text representation format. Secondly, all resources share unified interfaces and use HTTP Methods with explicit semantics to manipulate resources. The addition, deletion, modification, and query of resources are characterized by POST, DELETE, PUT, and GET. Simultaneously, it follows the HATEOAS (Hypermedia as the engine

of application state) principle to implement links between resources to support deeper interactions between clients and servers [8]. Although it is challenging to strictly adhere to all the principles of REST in a practical development environment, it is evident that REST has driven a revolution in the field of Web service.

Breaking free from cumbersome technical standards, RESTful Web Services are more concise and lightweight compared to SOAP-based Web Services. However, in contrast to SOAP-based Web Services, which focus on the discovery and composition centered around operations, the principles adhered to by REST Web Services introduce some challenges to their discovery and composition [9–11]. In the existing work, service discovery approaches can be broadly classified into two categories, keyword-based approaches [12, 13] and semantic-based approaches [14–16]. Semantic-based methods increase the machine's comprehensibility and improve the accuracy and recall rate of service matching by introducing ontology knowledge into the service discovery process. Service composition can be divided into two research directions: QOS-based composition [17, 18] and function-based composition [19–21]. The former considers the assurance of service quality in the composition process, while the latter considers combining several services with single function to achieve more complex functions. Our work belongs to the latter research direction. The function-based composition approach can be further divided into two types. One is the static composition method represented by BPEL (Business Process Execution Language) [22], which pre-constructs the composition workflow model and subsequently sends it to the execution engine to execute. The other is the dynamic composition approach represented by the graph model and the Petri net model [23, 24]. Without defining the composition process in advance, the service composition schemes can be automatically generated among many service nodes according to the requester's needs. Our work is a dynamic composition approach.

In this paper, we are inspired by existing methods for SOAP-based Web Service discovery and composition. By fully analyzing the characteristics of RESTful Web Service and combining semantic techniques, we propose a framework for implementing discovery and automatic composition of RESTful Web Services. Figure 1 is an overview of the proposed framework.

Firstly, regarding the RESTful Web Service description, we propose to expand the existing RESTful Web Service description specification — OpenAPI specification (OAS) by adding the *Resources* object to better accommodate REST characteristics. Subsequently, we conduct semantic annotation on the entity concepts that can best reflect the service functional characteristics in the expanded OAS description, so that they can be mapped to the semantic concepts in the related domain ontology, so as to support semantic-based service matching discovery and automatic composition.

After the service provider completes the syntactic description and the semantic annotation of key concepts for services, the services enter the registration center. Whether automatically searched and extracted by a crawler agent or registered by the provider, the services entering the registry should not be just scattered and unrelated service nodes. Such a storage pattern can greatly hinder the discovery efficiency of service composition schemes when considering the composition schemes based on the relationship between services. Therefore, we propose a RESTful Web Service

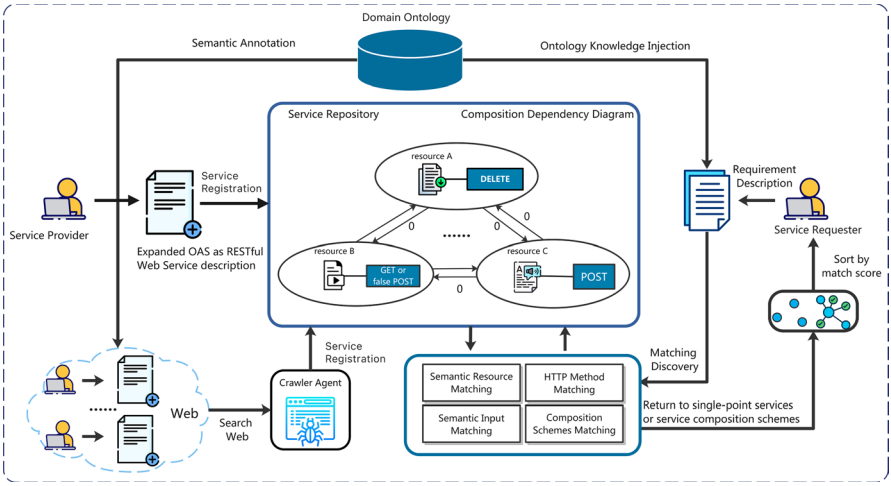


Fig. 1 Framework overview

composition dependency construction method that adapts to REST characteristics, allowing services to pre-generate service composition dependency diagrams during registration to support the generation of composition schemes in the service discovery process.

Finally, the service requester creates a demand description, and then the framework realizes the matching discovery between the demand description and service description information in the registry based on the common ontology knowledge of both sides. The matching result can be single-point services or service composition schemes, both quantified into matching scores.

The paper is organized as follows: Sect. 2 discusses the related issues of RESTful Web Service description. Section 3 proposes a RESTful Web Service composition dependency construction method that adapts to the characteristics of REST. Section 4 gives a discovery method for RESTful Web Services including composition schemes. Section 5 applies the proposed methods to experiment with RESTful Web Services in different domains, followed by analysis and summarization.

2 RESTful web service description with semantics attached

The service description is the foundation of Web Service applications. In this paper, we have chosen the OpenAPI Specification (OAS),[1] which is currently widely adopted, as the syntax description specification for RESTful Web Services [25, 26]. However, since OAS is designed for human users, it has not fully adapted to REST features and lacks support for automatic service discovery and composition.

[1] https://swagger.io/specification/.

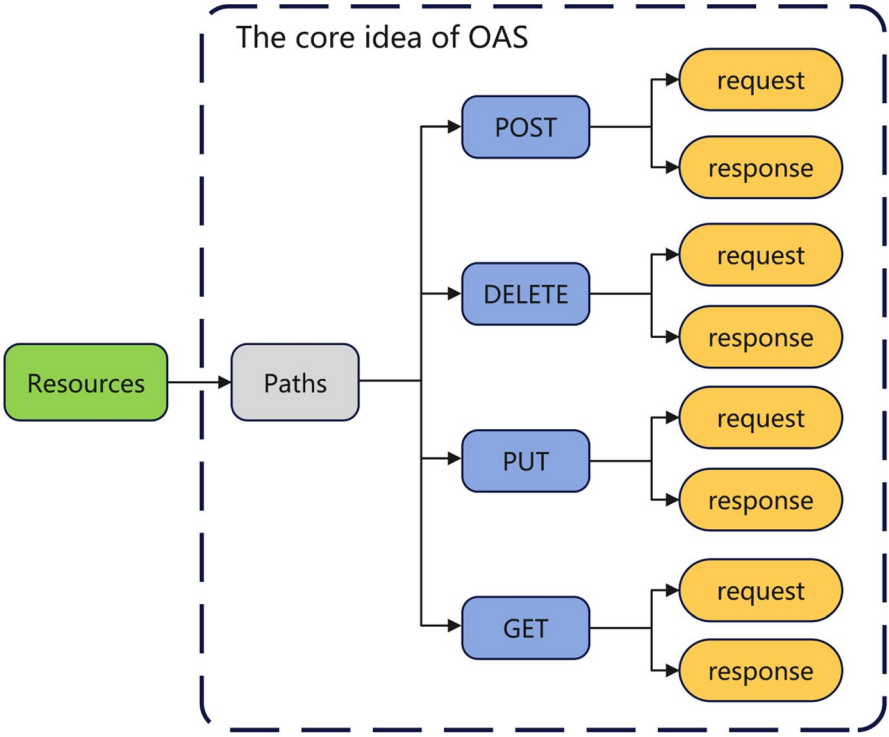


Fig. 2 The core idea of the expanded OAS

To address this, we extend the *Resources* object for OAS in Sect. 2.1. The expanded OAS comprehensively describes a RESTful Web service at three levels. Here, we actually regard a RESTful Web service as an operation endpoint manipulating resources through bound HTTP method. Subsequently, we extend the *semantic-linkTo* attribute for OAS in Sect. 2.2, achieving annotation of entity concepts in OAS to semantic concepts in domain ontology, to support semantic-based matching discovery and automatic composition in Sects. 3 and 4.

2.1 Expanded OpenAPI specification for RESTful web service description

OAS is both a standardized and programming language-independent RESTful API description specification, and a powerful RESTful API description framework supported by a range of tools (Swagger editor, API interfaces with other frameworks like Node.js, code generation, etc.). OpenAPI 3.0.0 is the first official version of the OpenAPI Specification, which is supported by Google, Microsoft, IBM, Oracle, and many other well-known enterprise users.

OAS defines rich objects and properties in JSON or YAML format. The core idea of OAS, represented by the dotted line in Fig. 2, is essential to describe a set of operation endpoints bound to the URL Path around request and response (i.e., input

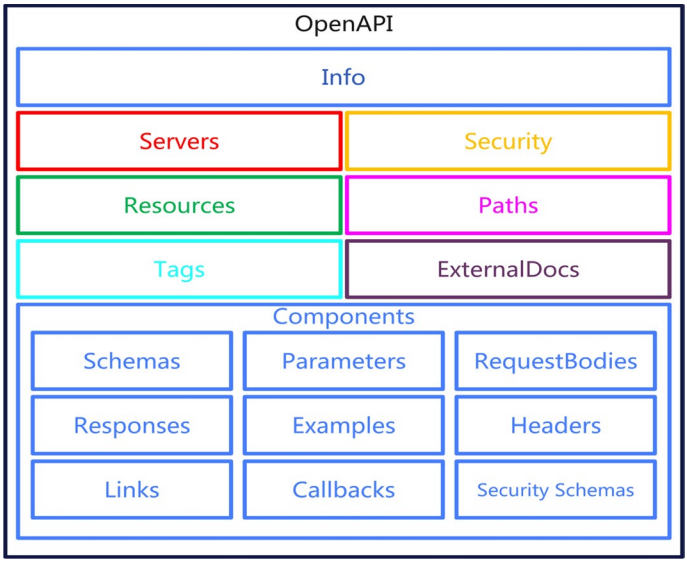


Fig. 3 The document structure of the expanded OAS

and output). This operation-centric description approach is indeed simple and clear for human users, but we believe that it has three shortcomings for the automatic discovery and composition of services in our work: Firstly, it does not quite meet the resource-oriented constraint of REST, and the representation of the resources is limited in the URL path. Secondly, using input and output as service function representation weakens the semantics inherent in HTTP verbs — adding, deleting, modifying, and querying resources are represented by POST, DELETE, PUT, and GET, respectively. Thirdly, the request and response of each operation endpoint often only reflect a partial representation of the resources being operated on and may not even reflect the resources being operated on. This is particularly evident on operation endpoints bound to the DELETE method. For example, in a geographic service, when using an operation bound to the DELETE method to delete a punctuation mark on the map, the request for that operation is likely to only include the *id* of the punctuation mark to be deleted, and the response is likely to only include the status code and related textual description information indicating whether the deletion was successful. This lack of resource representation hinders the envisioned process of matching discovery and automatic composition for our services. Therefore, we propose to expand the *Resources* object on the basis of the original OAS to improve these problems.

The extended *Resources* object is optional for operation endpoints bound with the GET method because their requests and responses already contain sufficient resource information. In this case, it serves as a supplementary means to some extent for resource representation. However, for semantically clear POST, PUT, and DELETE methods, the *Resources* object is necessary to compensate for their lack of explicit resource representation. The core idea of the expanded OAS, as shown

Table 1 The Key fields of the *Resources* object

Field name	Type	Description
Name	String	REQUIRED. The name of the resource
Description	String	A brief description of the resource
Content	Map[string, MediaType Object]	REQUIRED. Content of resource representation
paths	String	REQUIRED. Operation paths of current resource binding

```
resources:
- name: geocoding
  description: The current resource is a geocoding resource corresponding to the longitude and latitude of the
coordinate point converted from structured address data
  content:
    application/json:
      schema:
        properties:
          location:
            type: object
            properties:
              lat:
                type: number
                format: float
              lng:
                type: number
                format: float
          precise:
            type: integer
            description: Describe the marking accuracy
            format: int32
  paths: /geocoding   GET
```

Fig. 4 The example of resource description

in Fig. 2, actually regards a RESTful Web service as an operation endpoint manipulating resources through bound HTTP method. The document structure of the expanded OAS is shown in Fig. 3, which only expands the *Resources* object and its related properties, and inherits the original usage of other objects.

The *Resources* object has key fields as shown in Table 1. The *name* field is used as a mandatory option to name the current resource. The *description* field provides a brief description of the resource for the user. The *content* field, as a mandatory option, implements the representation of the resource by reusing the *MediaType* object that is used to describe the request and response content in OAS. The *paths* field is a mandatory option to bind the resource description to the defined operation endpoints. An example of a geocoded resource description is shown in Fig. 4.

In fact, the extended OAS completely describes a RESTful Web service on three levels:

1. How are the resources manipulated by the service represented? — For services bound with the GET method, it is represented by the service's response(output) or the *Resources* object. For services with explicitly semantic POST, PUT, and DELETE methods, it is represented by the *Resources* object.
2. What action do you want to perform on the resources? POST? DELETE? PUT? or GET?
3. How to perform this operation? — represented by the request(input) of each service.

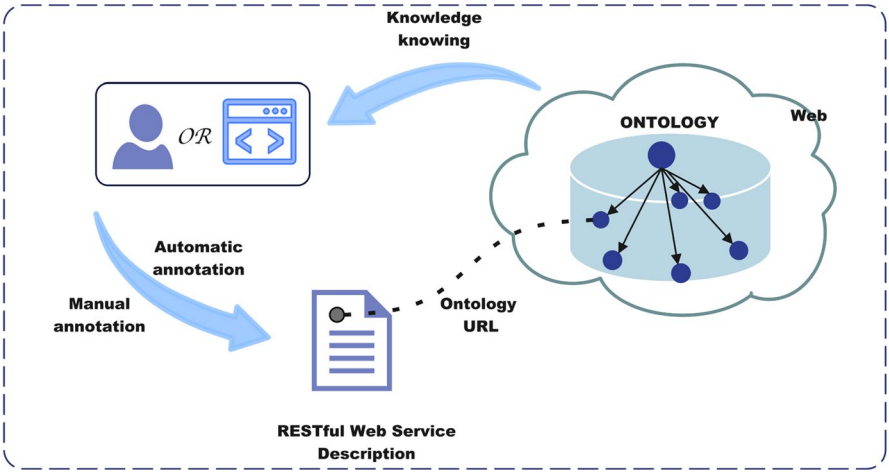


Fig. 5 The process of introducing ontology knowledge

The above constitutes a three-level description of RESTful Web Service in our understanding.

2.2 Semanticization of expanded OpenAPI specification

The expanded OAS description document provides a three-level syntax description for RESTful Web Service, but the syntax-level description can only achieve service discovery and composition from the perspective of keyword matching. Keyword matching introduces semantic ambiguity and uncertainty, and computers find it challenging to understand the semantic relationships between different concepts. In this regard, as shown in Fig. 5, introducing ontology knowledge to achieve the mapping of concepts in the description document to concepts in the ontology is a good solution to eliminate the aforementioned impacts. Ontology, as a formal expression of a set of concepts and their relationships in a specific domain, can be used to shield knowledge heterogeneity, improve matching accuracy, and enhance the relevance of concepts, thus increasing matching recall. In this scenario, as long as the requester permits, more services that are different from but relevant to requester requirements can be returned to the requester.

So how can we attach semantics to the OAS document? For this purpose, we extend the *semantic-linkTo* attribute to specify the association between the elements in OAS and the concepts in the ontology model. The application scope of the *semantic-linkTo* attribute is shown in Table 2. Semantic links pointing to ontology concepts can be attached to key elements in the *Resources* Object, the *Parameters* Object, the *Request Body* Object, the *Response* Object and the *Schema* Object. The “key elements" here refer to the concepts and attributes in OAS that best reflect the current service resource, request and response characteristics. In the environment of the Internet, each ontology model can be uniquely identified by an URL. When using the *semantic-linkTo* attribute, you only need

Table 2 The application scope of the *semantic-linkTo* attribute

Property	Applies to	Explanation
semantic-linkTo	Resources object	Realize semantic annotation of resource name and resource representation content
	Parameters object	Realize semantic annotation of parameter name and specific parameter content
	Request body object	Realize semantic annotation of request body content
	Response object	Realize semantic annotation of response content
	Schema object	Realize semantic annotation of properties in Schema

```
responses:
  '200':
    description: successful operation
    content:
      application/json:
        schema:
          properties:
            restaurant_name:
              type: string
              semantic-linkTo: http://example.com/geography#restaurant
            status:
              type: integer
              semantic-linkTo: http://example.com/commerce#opening_hours
              format: int32
            location:
              type: object
              semantic-linkTo: http://example.com/geography#location
              properties:
                lat:
                  type: number
                  semantic-linkTo: http://example.com/geography#latitude
                  format: float
                lng:
                  type: number
                  semantic-linkTo: http://example.com/geography#longitude
                  format: float
```

Fig. 6 The example of the *semantic-linkTo* attribute usage

to specify the corresponding ontology concept in the ontology model through the URL, and the final expression form is “URL # concept". As shown in Fig. 6, it is an application example of the *semantic-linkTo* attribute.

3 The construction method for RESTful web service composition dependencies

After the syntactic description and semantic annotation are completed by the service provider, the service should be registered in the repository. The single-point storage mode of services in the registration center significantly impedes the efficiency of discovering service composition schemes. Therefore, we propose to pre-generate a service composition dependency graph at service registration based on the resources and the request-response (i.e., input–output) relationships of the operations, which is used to support the discovery process of service composition schemes we discuss in the next chapter. In order to generate the composition dependency graph, we proposed the RESTful Web Service composition

rule models in Sect. 3.1 that conform to the REST characteristics, and then proposed a composition probability calculation method for RESTful Web Services in Sect. 3.2.

3.1 The composition rule models for RESTful web services

Most of the existing research oriented to service function composition methods is for SOAP-based Web Services, and relatively little research has been done for RESTful Web Services. In the operation-centered perspective of SOAP-based Web Services, the core ideas of its service composition methods can be summarized into the following rules:

- **Operation composition rule 1:** The input and output sets of operation A respectively are IN_a and OUT_a. The input and output sets of operation B respectively are IN_b and OUT_b. Then operation A and operation B can be composited into operation C. The input set of operation C: $IN_c = IN_a \cup IN_b$, the output set: $OUT_c = OUT_a \cup OUT_b$, which is written as $A + B = C$.
- **Operation composition rule 2:** When one concept in the output set OUT_a of operation A is matched with another concept in the input set IN_b of operation B, there is possibility of service composition between operation A and operation B, which is written as $A \rightarrow B$.

The above two rules reflect the general law of parallel and serial composition between services, which naturally fits the perspective of operation considering only input and output. But for RESTful Web Services, such a composition thought does not take its resource-oriented characteristics into account, and lost the semantics of HTTP methods POST, DELETE, PUT and GET as well, which is not completely suitable for RESTful Web Services. Therefore, based on the cognize that “a RESTful Web service is regarded as an operation endpoint manipulating resources through bound HTTP method", through comprehensive research on the thousands of RESTful APIs in the Rapid API Hub[2] and a thorough analysis of REST characteristics, we propose the service composition rule models for RESTful Web Services.

Before introducing the composition rule models, there is something that needs clarification. In Rapid API Hub, we observed an interesting phenomenon: due to security or communication overhead considerations, many service providers choose to use the POST method instead of the GET method to get resources. In this case, the POST method does not strictly adhere to the semantics of resource creation. We refer to this method as “FALSE POST". This is a practical consideration in our development because REST is not a strict standard and may not always be strictly adhered to. In this context, the rule models are as follows:

1. Homogeneous self-increment rule: Services bound to semantically similar HTTP methods can naturally compose a larger-grained service in parallel(as shown

[2] https://rapidapi.com/hub.

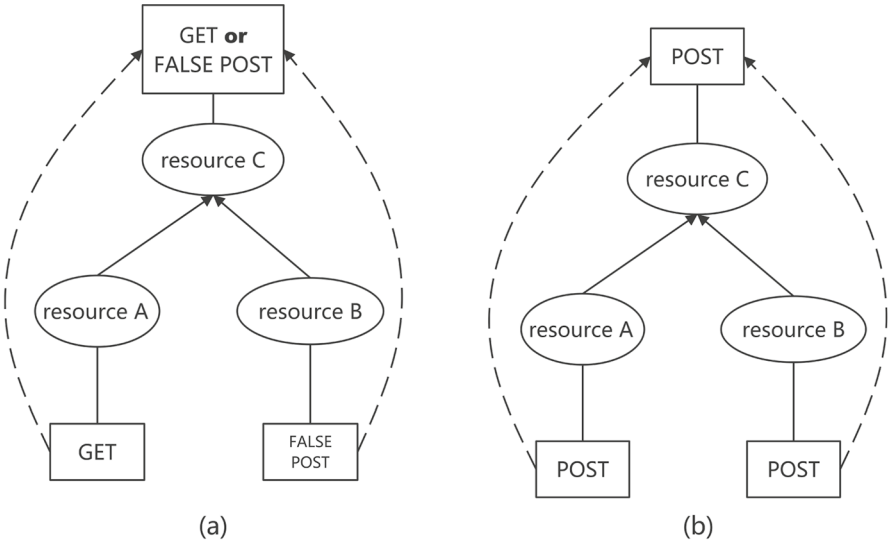


Fig. 7 Homogeneous self-increment rule

in Fig. 7). Both the GET method and the FALSE POST method share the semantics of getting resources. Services bound to these two HTTP methods can be composed in parallel, while services bound to POST, PUT, and DELETE methods can only be composed in parallel with services using the same HTTP method. The resources manipulated, required inputs and outputs of the composite service are collectively provided by the sub-services.

2. GET no blocking rule: If we regard a RESTful Web service as a “black box", the GET method is undoubtedly the most special one among the four methods to operate the resources inside this “black box". Because only the GET method semantically gets something from the “black box" without changing the state of the resources in the box. While the other three methods (POST, DELETE, and PUT) change the state of the resources inside the box by inputting information. According to this feature, on the basis of the operation composition rule 2, we make the rules of RESTful Web Service heterogeneous HTTP method composition(shown in Fig. 8).

Based on the specificity of the GET method, the output of operations bound to the GET method can provide inputs to operations bound to the POST, DELETE, and PUT methods, but conversely, operations bound to the POST, DELETE, and PUT methods cannot provide inputs to other operations, regardless of what HTTP methods they are bound to. The FALSE POST method, which also signifies resource retrieval, follows the same rule as the GET method.

This rule is quite reasonable. We can consider that in the actual production environment of REST APIs. The operation bound with POST, DELETE, and PUT methods will not include the information related to the functional characteristics of this operation in the outputs, and often only return some prompt messages, such as “Created successfully", “Deleted successfully" and “Modified successfully", which cannot be used as inputs to another operation. In the final analysis, the action attribute

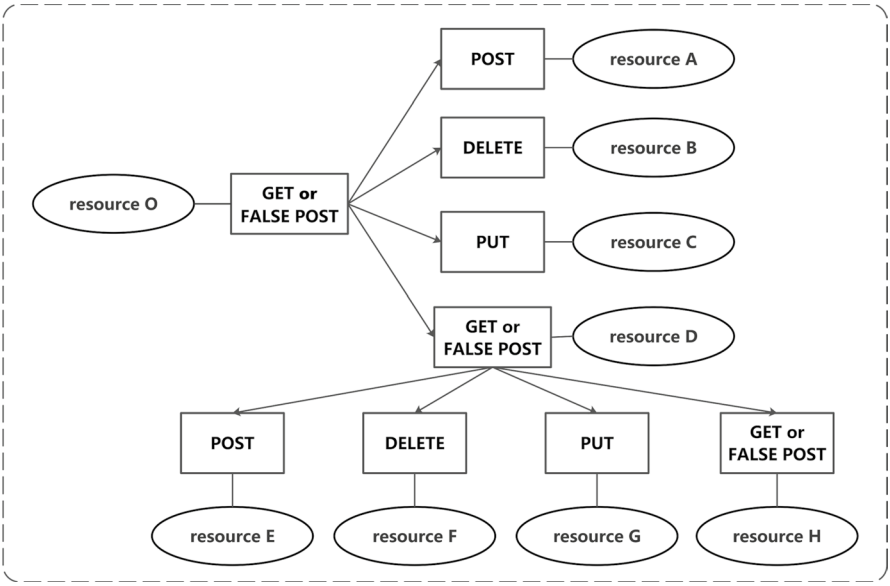


Fig. 8 GET no blocking rule

of these three methods is too strong and the information attribute is too weak. Therefore, a service composition chain will be cut off when it encounters the operations bound with POST, DELETE, and PUT methods, and only operations bound with the GET method or the FALSE POST method have the possibility to be composed all the time, so it is called “the GET no blocking rule".

For the GET no blocking rule shown in Fig. 8, when a concept in the output set of an operation establishes a matching relationship with a concept in the input set of another operation, the two services have the possibility to be composed. Then how to establish the matching relationships between the outputs and inputs of the operation? And how to quantify the composition possibility between services for comparison?

3.2 A composition possibility calculation method for RESTful web services

Now, let’s discuss how to establish matching relationships for the outputs and inputs of different operations under the GET no blocking rule. In the previous discussion in Sect. 2.2, we have already known that keyword-based matching methods lead to semantic ambiguity and challenges in determining conceptual relationships. Therefore, based on the semantic OAS in Sect. 2.2, using the ontology semantic similarity calculation method, we calculate the semantic similarity of ontology concepts annotated on outputs and inputs in different service OAS descriptions, so as to obtain the semantic matching score between an output of one operation and an input of another operation.

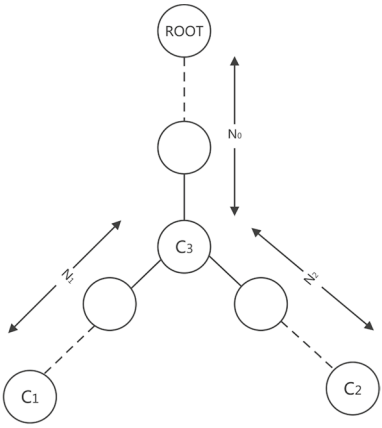


Fig. 9 The *WuAndPalmer* algorithm calculation principle

For the calculation of semantic similarity between ontology concepts, we have adopted the *WuAndPalmer* algorithm, which is widely used in several studies [27, 28]. This algorithm is a classic approach based on semantic distance to compute the semantic similarity of ontology concepts. The reason for selecting this algorithm lies in its generality across different ontologies and its simplicity. While the structural characteristics may vary among different ontologies, the hierarchical nature inherent in ontologies is universal. Therefore, this algorithm can enhance adaptability to ontologies with different characteristics. The algorithm calculates semantic similarity using the positional relationship between two concepts and their nearest common parent concept, and the calculation principle is shown in Fig. 9.

Given an ontology Ω, *ROOT* is the root concept in Ω, C_1 and C_2 are any two concept nodes in Ω, C_3 is the nearest common parent concept node of C_1 and C_2, N_o is the number of nodes on the path from C_3 to *ROOT*, N_1 is the number of nodes on the path from C_1 to C_3, and N_2 is the number of nodes on the path from C_2 to C_3, then the semantic similarity matching score $SimCalc(C_1, C_2) \in [0, 1]$ between C_1 and C_2 is defined in Eq. (1).

$$SimCalc(C_1, C_2) = \frac{2 \cdot N_0}{N_1 + N_2 + 2 \cdot N_0} \tag{1}$$

With the ontology semantic similarity calculation method, we can get the semantic matching score between a concept in the output set of one operation and a concept in the input set of another operation under the GET no blocking rule. So on this basis, how to get the overall matching score between the output set and the input set of two operations, and then get the quantitative index of the possibility of composing two RESTful Web services? To this end, we propose a weighted modified approach to quantify the possibility of composing RESTful Web Services, which is based on the following principles:

Given two Web services WS_1 and WS_2. The set consisting of the ontology concepts attached to outputs of WS_1 is *Out*, and the number of elements in set *Out* is m_1. The set consisting of the ontology concepts attached to inputs of WS_2 is *In*, and the

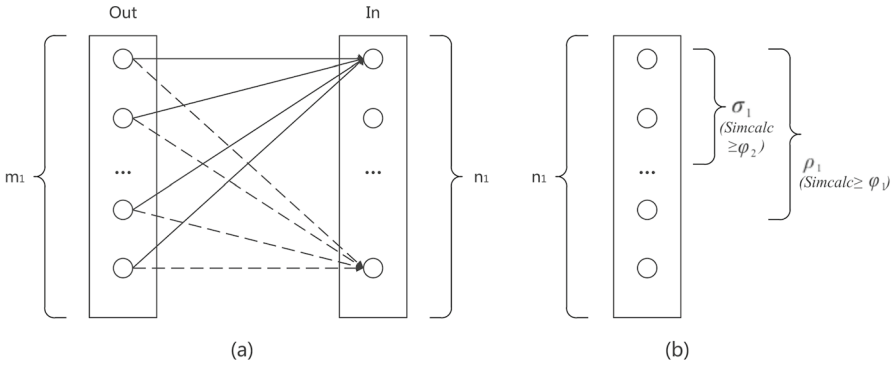


Fig. 10 The Principle of output-input matching

number of elements in the set *In* is n_1. Then the possibility of composing the operations of WS_1 and WS_2 $OpCompoScore \in [0,1]$ is defined in Eq. (2a).

(2a) is derived from a two-part weighted summation. The first half is the overall matching score obtained by computing *SimCalc* for the set *Out* of WS_1 against the set *In* of WS_2. Because in the output-input matching process of WS_1 and WS_2, the number of inputs of WS_2 must be satisfied first. We can accept that the number of elements m_1 in the *Out* set of WS_1 is greater than the number of elements n_1 in the *In* set of WS_2, but we do not want the operation of WS_2 to have some inputs not being satisfied. Therefore, the overall matching score is to match the set *Out* of WS_1 according to the set *In* of WS_2, and then divided by the number of elements n_1 of the set *In*, the principle diagram is shown in Fig. 10a.

$$OpCompoScore(WS_1, WS_2) = \alpha_1 \cdot \frac{\sum_{j=1}^{n_1} \max_{i=1}^{m_1}(SimCalc(Out_i, In_j))}{n_1} + (1-\alpha_1)\cdot\theta_1 \tag{2a}$$

$$\theta_1 = \beta_1 \cdot \frac{\rho_1}{n_1} + \gamma_1 \cdot \frac{\sigma_1}{\rho_1} + \delta_1 \cdot \frac{2}{\pi} \arctan\left(\frac{\sqrt{\rho_1}}{\sqrt{\sum_{k=1}^{\rho_1}(\max_{l=1}^{m_1}(SimCalc(Out_l, In_k)) - u_1)^2}} - 1\right) \tag{2b}$$

$$\mu_1 = \frac{\sum_{k=1}^{\rho_1} \max_{l=1}^{m_1}(SimCalc(Out_l, In_k))}{\rho_1} \tag{2c}$$

The second half of (2a) is a weighted correction factor θ_1 ($\theta_1 \in [0, 1]$ see Eq. (2b)) for data features not exposed in the overall matching score which is used to improve or reduce the competitiveness of WS_2 and other services relative to WS_1. θ_1 is also derived from the weighted sum of the three components with weights

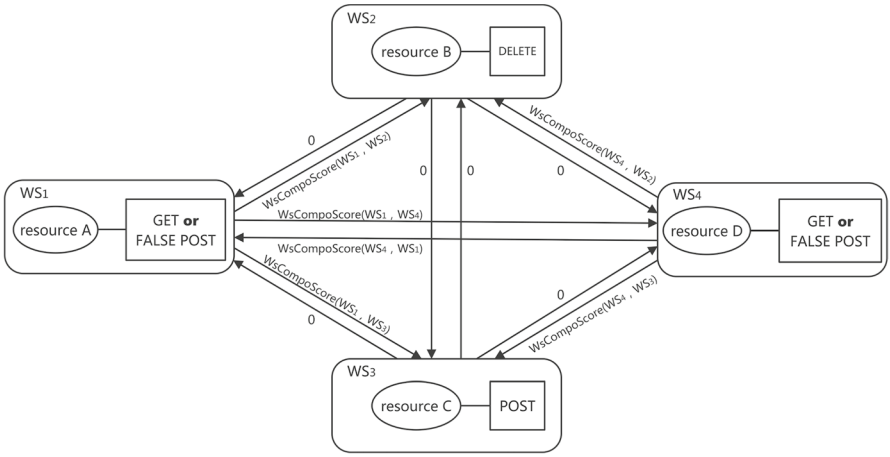


Fig. 11 The service composition dependency diagram

(the schematic diagram is shown in Fig. 10b), $\beta_1 + \gamma_1 + \delta_1 = 1$. In the first part, ρ_1 is the number of concept element In_j whose $\max_{i=1}^{m_1}(SimCalc(Out_i, In_j)) \geq \varphi_1$ in the set In of WS_2. φ_1 is the minimum threshold we set that can be considered as "successfully matched" between the output and input ontology concepts. Then $\frac{\rho_1}{n_1}$ represents the proportion of elements that can be considered as successfully matched in the set In to the whole set. In the second part, σ_1 is the number of concept elements In_j whose $\max_{i=1}^{m_1}(SimCalc(Out_i, In_j)) \geq \varphi_2$ in the ρ_1 matched successfully concept elements of the set In. φ_2 is the minimum threshold we set that can be regarded as "high-quality matching" between the output and input ontology concepts. Then $\frac{\sigma_1}{\rho_1}$ characterizes the proportion of high-quality matching concepts among the successfully matched concept elements. The third part represents the dispersion of the $\max_{i=1}^{m_1}(SimCalc(Out_i, In_j))$ fraction of ρ_1 successfully matched concept elements in the set In. We use the inverse of the standard deviation of the $\max_{i=1}^{m_1}(SimCalc(Out_i, In_j))$ fraction of ρ_1 matched successfully concept elements as the measuring standard, and the normalization is performed using the function $f(x) = \frac{2}{\pi} \cdot \arctan(\frac{1}{x} - 1)$ to obtain a value between [0,1]. μ_1 is the mean of the $\max_{i=1}^{m_1}(SimCalc(Out_i, In_j))$ fraction of ρ_1 successfully matched concept elements (see Eq. (2c)).

Using the above formula, we can get the composition probability score between the operation of WS_1 and the operation of WS_2. After that, under the constraint of the GET not blocked rule, the final quantitative service composition possibility score $WsCompoScore \in [0, 1]$ of WS_1 and WS_2, is defined in Eq. (3). In this equation, $A_1 = 1$ when the operations of WS_1 and WS_2 satisfy the GET no blocking rule, and $A_1 = 0$ otherwise. In this way, oriented to RESTful Web Services, under the constraint of the GET no blocking rule, the operations owned by each Web service are connected as a weighted directed graph by matching

outputs and inputs, and the weight of each edge in the graph is provided by the quantitative score of the composition possibility between two services, as shown in Fig. 11. The final result is a service composition dependency graph that should be pre-generated during service registration, which provides the foundation for us to discover the service composition schemes to meet requesters' needs later.

$$WsCompoScore(WS_1, WS_2) = A_1 \cdot OpCompoScore(WS_1, WS_2) \tag{3}$$

4 The discovery method for RESTful web services including composition schemes

After the expanded OAS description, semantic attachment and registration storage, RESTful Web services are waiting for service requesters to discover and invoke them in the service repository in the form of a service composition dependency graph. So how does the service requester build the requirement description and how does the service requester's requirement description accurately match the service description information in the service repository? Meanwhile, the services matched by service discovery can be single-point services or composition schemes provided by multiple service nodes. So how to determine the service composition schemes and recommend them to the requesters together with the single-point services?

4.1 The description of requester requirements and the discovery for single-point services

The requesters' requirement description can be characterized by a quadruple: < *target(Optional), resource, HTTP method, operation*>. The specific description content of the tuple is provided by the ontology knowledge shared by both the service provider and the service requester, supporting the service matching discovery based on semantic similarity.

resource represents the resources manipulated by the intent service. *HTTP method* represents the HTTP action of the intent service. *operation* represents what the operation of the intent service should look like by describing the inputs of the intent service. These three elements actually correspond one-to-one with the three layers of service description. The optional element, *target*, is specifically prepared for services bound to explicit semantic POST, PUT, and DELETE methods. Through the analysis of services in Rapid API Hub, we found that services bound to these three methods are often deeply integrated with the platforms they belong to. This aligns with our previous analysis that, unlike GET or FALSE POST methods, creating, modifying, and deleting resources will change the state of the server, which makes services bound to the three methods of POST, PUT and DELETE deeply embedded within the platform, as shown in Fig. 12. Therefore, to discover such services, it is crucial to specify the *target*. The *target* could be the platform name, application

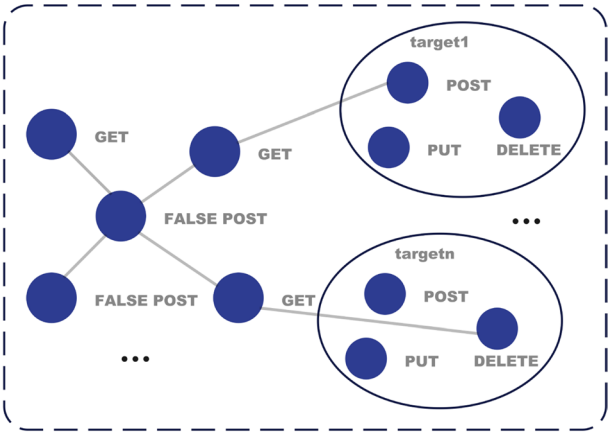


Fig. 12 POST, PUT, and DELETE methods deeply embedded within the platform

name, organization name, and so on. For example, when a requester wants to find an API that allows posting a new entry on a platform like Wikipedia, the first thing they need to determine is the "Wikipedia" target.

With the requester requirement description, we next discuss how to match the requester requirement with the service description information. As mentioned earlier, keyword matching is not reliable, so we propose a method for calculating the matching score of RESTful Web Services based on semantic technology, which is calculated as follows:

Given a service *WS* and a requirement *RE*, the set of ontology concepts attached to the resource descriptions of the *WS* is *RS*. The HTTP method of *WS* is *MethodS*, and the set of ontology concepts attached to the input descriptions of the *WS* is *IS*, then $WS = < RS, MethodS, IS >$. The set of ontology concepts used in the resource requirement part of the requirement *RE* is *RR*, the HTTP method of the requirement is *MethodR*, and the set of ontology concepts used in the input requirement part is *IR*, then $RE = < RR, MethodR, IR >$. The calculation of the matching score between *WS* and *RE* consists of three processes.

1. Semantic resource matching: Matching resources manipulated by intent services. Assuming that the number of elements in *RS* is m_2 and the number of elements in *RR* is n_2, then the matching score $ReScore(RS, RR) \in [0, 1]$ between *RS* and *RR* is defined in Eq. (4a). Because whether the output-input composition matching score between services we discussed in Sect. 3.2 or the resource and input matching scores between services and requirements that we will discuss next, they are essentially cross-matching problems between two different sets based on the semantic similarity calculation method *SimCalc*. Therefore, the Equation for the matching discovery score of resource and input is similar to the one we proposed in Sect. 3.2, and we only explain it briefly.

In the resource matching process, the number of requested resources in *RR* should be satisfied first. We can accept that the number of elements m_2 of the set *RS* is greater than the number of elements n_2 of the set *RR*. In other words, we do not want

to see the situation that the resources requested by requesters are not satisfied. So in Eq. (4a), the denominator is the number of elements n_2 of the set *RR*. The calculation of the correction factor in Eqs. (4b), (4c) is also performed for the *RR* set of *RE*. ρ_2 and σ_2 are the concepts of successful matching and high-quality matching in the set *RR*, respectively.

$$ReScore(RS, RR) = \alpha_2 \cdot \frac{\sum_{j=1}^{n_2} \max_{i=1}^{m_2}(SimCalc(RS_i, RR_j))}{n_2} + (1 - \alpha_2) \cdot \theta_2 \quad (4a)$$

$$\theta_2 = \beta_2 \cdot \frac{\rho_2}{n_2} + \gamma_2 \cdot \frac{\sigma_2}{\rho_2} + \delta_2 \cdot \frac{2}{\pi} \arctan\left(\frac{\sqrt{\rho_2}}{\sqrt{\sum_{k=1}^{\rho_2}(\max_{l=1}^{m_2}(SimCalc(RS_l, RR_k)) - u_2)^2}} - 1\right) \quad (4b)$$

$$\mu_2 = \frac{\sum_{k=1}^{\rho_2} \max_{l=1}^{m_2}(SimCalc(RS_l, RR_k))}{\rho_2} \quad (4c)$$

2. HTTP method matching: Matching the HTTP actions for the intended services. When *MethodS* of *WS* is the same as *MethodR* of *RE*, the matching score between them is $A_2 = 1$, otherwise $A_2 = 0$. It is worth noting that both the GET method and the FALSE POST method, which represent resource retrieval, are two methods that are semantically consistent.

3. Semantic input matching: The input represents the "cost" the requester is willing to pay for using the service, and matching on the input essentially reflects the requester's expectations on how the intended service should be utilized. Assuming that the number of elements in *IS* is m_3 and the number of elements in *IR* is n_3, then the matching score $IScore(IS, IR) \in [0, 1]$ between *IS* and *IR* is defined in Eq. (5a).

In the input matching process, the number of service inputs in *IS* should be satisfied first. We can accept that the number of elements m_3 of the set *IS* is less than the number of elements n_3 of the set *IR*. In other words, we do not want to see the services returned to the requesters with inputs that the requesters do not expect. So in Eq. (5a), the denominator is the number of elements m_3 of the set *IS*. The calculation of the correction factor in Eqs. (5b), (5c) is also performed for the *IS* set of *RE*. ρ_3 and σ_3 are the concepts of successful matching and high-quality matching in the set *IS*, respectively.

$$IScore(IS, IR) = \alpha_3 \cdot \frac{\sum_{i=1}^{m_3} \max_{j=1}^{n_3}(SimCalc(IS_i, IR_j))}{m_3} + (1 - \alpha_3) \cdot \theta_3 \quad (5a)$$

$$\theta_3 = \beta_3 \cdot \frac{\rho_3}{m_3} + \gamma_3 \cdot \frac{\sigma_3}{\rho_3} + \delta_3 \cdot \frac{2}{\pi} \arctan\left(\frac{\sqrt{\rho_3}}{\sqrt{\sum_{l=1}^{\rho_3}(\max_{k=1}^{n_3}(SimCalc(IS_l, IR_k)) - u_3)^2}} - 1\right) \quad (5b)$$

$$\mu_3 = \frac{\sum_{l=1}^{\rho_3} \max_{k=1}^{n_3}(SimCalc(IS_l, IR_k))}{\rho_3} \tag{5c}$$

After three matching steps, the final matching score $Discovery- Score(WS, RE) \in [0, 1]$ between WS and RE is defined in Eq. (6). So far, the similarity comparison between the services in the service repository and the requesters' requirements is converted to a comparison of the numerical size of the *DiscoveryScore*, so that we can return the services to the service requesters by *DiscoveryScore* size.

$$DiscoveryScore(WS, RE) = A_2 \cdot (\alpha_4 \cdot ReScore(RS, RR) + (1 - \alpha_4) \cdot IScore(IS, IR)) \tag{6}$$

4.2 The discovery for composition schemes

The results of service discovery can be either single-point services or service composition schemes. Based on the service composition dependency diagram obtained in Sect. 3 and our proposed service discovery approach, we can easily find several service composition scheme discovery approaches using the graph model-based approach. We present one of them here, which should include at least the following steps

1. Set the threshold η_1 to reject useless relationship edges in the service composition dependency diagram. When $WsCompoScore \geq \eta_1$, we can assume that the composition of the two services is more likely to be connected. That is to say, one service node can reach another service node. When $WsCompoScore \leq \eta_1$, we consider that the two service nodes are disconnected.
2. According to the requesters' requirement RR for resources, using Eq. (4a), select the services with $ReScore \geq \eta_2$(η_2 is the threshold set to judge that the resources of the service can satisfy or partially satisfy the requesters' requirements to a certain extent) and the semantically similar HTTP method. The resources manipulated by these services serve as sub-resources to satisfy the requester's needs, while these services serve as terminal services for the service composition route.
3. According to the requesters' requirement IR for inputs, using Eq. (5a), select the services with $IScore \geq \eta_3$(η_3 is the threshold set to judge that the inputs of the service can satisfy or partially satisfy the requesters' requirements to a certain extent). These services serve as starting services for the service composition route.
4. In the composition dependency diagram obtained by step 1 with the useless relationship edges removed, determine the service composition routes from the starting services to the terminal services. Firstly, graph search algorithms such as Depth First Search or Breadth First Search are used to search for the direct path from a starting point service to a terminal service as the "trunk road" of a composition route. Then, because the inputs of service nodes on the "trunk road" should be satisfied as much as possible, the services providing inputs for nodes

on the “trunk road" should also be added to this composition route as the “branch road" to complete the “trunk road". “Trunk road" and “branch road" constitute together a complete composition route from a starting service to a terminal service. Multiple service composition routes form a service composition scheme.

5. Identify additional inputs and outputs on the composition routes from the starting services to the terminal services due to the inexact output-input match. The additional inputs, together with the inputs of the starting services, serve as the inputs for the service cluster composed by the current service composition scheme. As for additional outputs, when the HTTP method requested by the requester is GET or FALSE POST, the additional outputs, along with the resources manipulated by the terminal services, serve as the manipulated resources for the service cluster composed by the current service composition scheme. When the HTTP method requested by the requester is POST, DELETE, or PUT, the additional outputs are discarded and have no effect on the discovery process.
6. Determine the overall resources, HTTP method, and inputs of the service cluster composed of services in the service composition scheme, and obtain the matching discovery score of the service composition scheme according to Eq. (6).

:

In summary, the discovery process of a service composition scheme is based on the requester’s requirements, employing a graph search algorithm to explore paths in the service composition dependency graph. Key steps include: 1. Identifying terminal sub-services through resource matching. 2. Identifying starting sub-services through input matching. 3. Searching for paths from starting sub-services to terminal sub-services. 4. Treating all services involved in the service composition scheme as a whole service. It’s worth noting that, during the search for composition paths, services that provide inputs to the sub-services on the composition path should also be included in the composition scheme. We have reason to believe that certain machine learning algorithms can enhance the efficiency and quality of discovering composition schemes.

5 Experiment and conclusion

For the expanded OAS and its semanticization, we have verified it in Sect. 2 by giving examples of expanded OAS describing RESTful Web Services. In order to conduct experimental analysis of the methods we proposed in Sect. 3 and Sect. 4, we need a set of RESTful Web services and domain ontologies that describe the services within the service set. As of our knowledge, there is currently no readily available service set suitable for directly testing our methods. Therefore, we take the real-world APIs in the Rapid API Hub as the main body, and by collecting and organizing the services from some service collection sites, we get a total of 1,338 RESTful Web services in the three domains: geographic information (441 services), healthcare (129 services), and financial economics (768 services). The selection of

Table 3 Four requests described using concepts from the domain ontologies

Request	Description	Resources	Method	Inputs
Request1	Get the temperature at the given coordinates	Temperature	GET	Longitude, latitude
Request2	Calculate calories burned	calory	GET	human_height, weight, distance, time
Request3	Stock trend prediction	market_trend	GET	stock
Request4	Upload BMI data on *MyFitnessPal*(target) platform	BMI	POST	human_height,weight

Table 4 The single-point services with with the highest matching score

Service	Description	Resources	Method	Inputs	Score
Weather Service	Get weather information	Weather	GET	Address	0.91
Energy Consumption1	Energy consumption calculation	Energy	GET	BMI,distance,time	0.93
Stock Information	Get stock information	economic_indicator	GET	stock	0.94
MyFitnessPal	*MyFitnessPal*'s POST method	BMI	POST	BMI	0.97

web services from multiple domains aims to demonstrate the adaptability of our methods.

Correspondingly, we have also chosen three domain ontologies. For the geographic information domain, we selected the *Semantic Web for Earth and Environmental Terminology (SWEET) Ontologies*,[3] which is an intermediate ontology covering approximately 6000 terms and concepts in Earth system science, demonstrating significant authority in the field of geographic information. In the domains of healthcare and financial economics, we have chosen concepts from the relevant domains within the general ontology repository *DBpedia*.[4] *DBpedia* is a widely recognized large-scale knowledge graph project, containing over 228 million entities. We extracted a portion of concepts from the healthcare and financial economics domains within *DBpedia*. We described the service set using concepts from these ontologies.

After obtaining the service set and domain ontologies for testing, we simulated and established 30 request descriptions. Before analyzing the matching results between these requests and services, we would like to demonstrate the specific effects of the discovery and composition methods through interesting practical cases. Considering space limitations, we selected four of the requests, see Table 3. For each request, we gave the single-point services with with the highest matching

[3] https://github.com/ESIPFed/sweet.

[4] https://www.dbpedia.org/.

Table 5 The composition scheme with the highest matching score

Composition schemes	Method	Inputs for each service	Outputs for each service	Composition possibilities	Discover score
Reverse geocoding Weather service	GET GET	Longitude, latitude address	Address weather	1.0	0.96
BMI calculation Energy Consumption2	GET GET	human_height, weight BMI, exercise_level	BMI energy	0.72	0.94
Stock Potential Calculator Stock information	GET GET	P/E,P/B, P/S stock	market_trend economic_indicator	0.97	1.0
BMI Calculation *MyFitness_Pal*	GET POST	human_height, weight BMI	BMI —	1.0	1.0

score and the composition scheme with the highest matching score. At this point, for the weight coefficients in the methods, it may not be the most accurate, but tentatively, $\alpha_1=\alpha_2=\alpha_3=0.8$, $\alpha_4=0.5$, $\beta_1=\beta_2=\beta_3=0.5$, $\gamma_1=\gamma_2=\gamma_3=0.3$, $\delta_1=\delta_2=\delta_3=0.2$. The threshold of successful matching $\varphi_1=0.5$, and the threshold of high-quality matching $\varphi_2=0.75$ in Eqs. 2, 4, and 5. Then, for other thresholds, we set two types: one is loose matching 0.5, and the other is strict matching 0.75. We will conduct data analysis on the matching results of these two situations later. Currently, the scores in Tables 4 and 5 adopt a loose matching of 0.5. Another point worth noting is that whether it is a request description or a service description, we are using concepts within the domain ontologies, but the URL prefix of the concept is hidden.

From the results in Table 4, it can be observed that even when the concepts in the request descriptions and service descriptions are heterogeneous, functionally similar services can still be identified through the semantic association between concepts. For example, obtaining *temperature* by *latitude* and *longitude* can be functionally similar to obtaining *weather* by *address*, showcasing the benefits of introducing ontology knowledge. Additionally, the inclusion of ontology knowledge brings precision to specific concepts, such as *BMI*, avoiding ambiguity. In Table 5, we also observe that ontology semantics contributes to the intelligence of the composition process and the discovery process of composition schemes. Due to the specificity of POST, DELETE, and PUT methods, let's delve into the composition process for Request 4. The goal of Request 4 is to find a service on the *MyFitnessPal* platform (the target is *MyFitnessPal*,[5] a health management platform) that can POST BMI by inputting *human_height* and *weight*. However, in the service repository, there is only a service that directly inputs *BMI* to create resources on this platform. At this point, there is another *BMI calculation* service in the repository that can calculate

[5] https://myfitnesspalapi.com/.

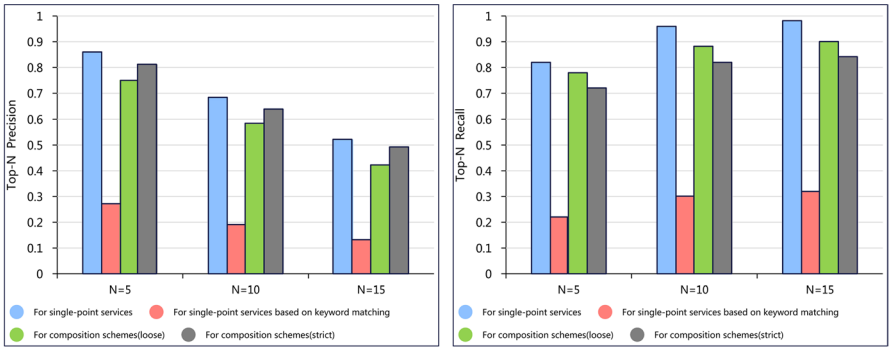


Fig. 13 The average *Precision* and average *Recall* of the top N services based on matching scores under four conditions

BMI based on *human_height* and *weight*. These two services naturally combine to provide the required functionality. Although the composition of these two services is based on the complete *BMI* match, we can also see in Composition Schemes 2 and 3 that the possibility of composition is enhanced through ontology semantics when the services are connected.

After understanding our interesting and real cases of discovery and composition, we simulated 30 different requests targeting the service set of 1338 services. To evaluate and analyze the matching situations, we need some evaluation metrics. Although our method is not a machine learning approach, we draw inspiration from it. For each request, we manually sought services in advance from all available services that could meet the requirements, thereby constructing a subjectively satisfying service set *Subjective*. Subsequently, the service set composed of the top N services based on scores obtained from our matching discovery method is set as the *Objective*. Therefore, for the current request, *Precision* among the top N services in terms of matching scores is defined in Eq. (7a), *Recall* is defined in Eq. (7b), and *F-measure* is defined in Eq. (7c). Taking the absolute value represents the quantity of services.

$$Precision = \frac{|Subjective \cap Objective|}{|Objective|} \tag{7a}$$

$$Recall = \frac{|Subjective \cap Objective|}{|Subjective|} \tag{7b}$$

$$F - Measure = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \tag{7c}$$

Precision reflects whether our method might make misjudgments for services that are not in the *Subjective* set, further indicating the reliability of our method. *Recall* reflects the method's ability to capture services in the *Subjective* set. We

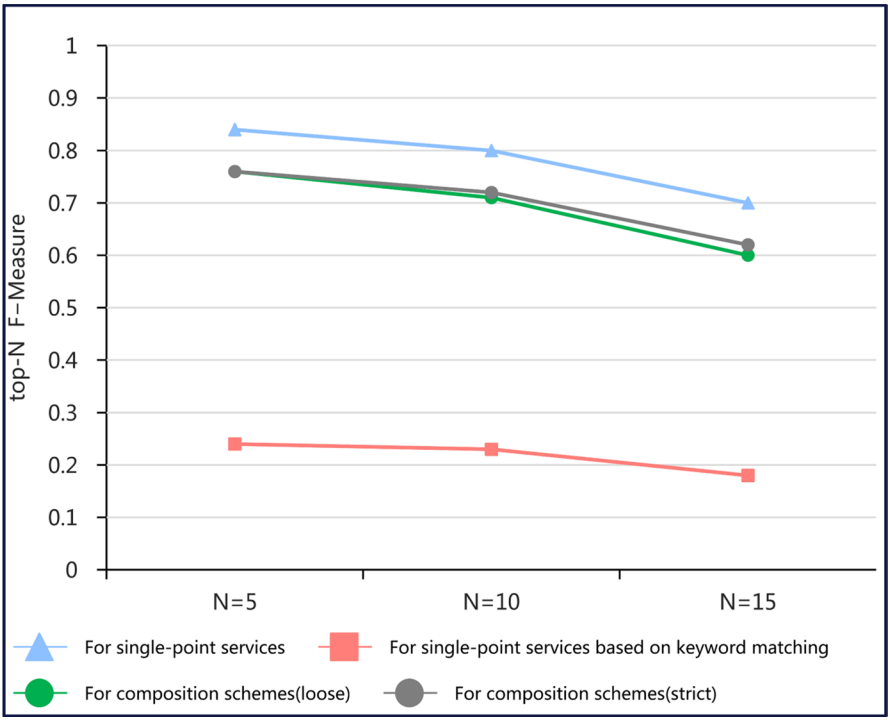


Fig. 14 The average *F-measure* of the top N services based on matching scores under four conditions

use the comprehensive metric *F-measure* of the two to reflect whether the service set objectively calculated by our method is consistent with our subjective beliefs. Due to the varying number of services in the *Subjective* set for each request, we chose N as 5, 10, 15 to cover all situations as much as possible. Eventually, we obtained the average *Precision* and average *Recall* for single-point service discovery, keyword-based single-point service discovery, loose-mode composition scheme discovery, and strict-mode composition scheme discovery for 30 requests, as shown in Fig. 13. Here, the *Precision* and *Recall* for composition schemes were also calculated based on the experimental approach mentioned above.

For *Precision* in Fig. 13a, firstly, we can observe that it decreases overall with the increase of N. This is mainly because as N increases, more services that are not in the *Subjective* set enter the matching results. This is also influenced by the varying number of services in the *Subjective* set for each request. Secondly, compared to the keyword-based single-point service matching method, our method shows a significant improvement in *Precision*. This is consistent with the results analyzed in the practical cases above, indicating that our method excels in discovering services that are semantically similar in functionality. Lastly, the *Precision* of composition scheme discovery in both loose and strict modes is lower than that of single-point service discovery but still maintains a relatively high level. This is mainly because graph-based composition scheme search introduces more

uncertainty. Additionally, The loose mode with a lower threshold exhibits lower *Precision* compared to the strict mode with a higher threshold.

For *Recall* in Fig. 13b, the overall trend shows the opposite behavior, increasing with the growth of N, although the increase is not substantial when N=15. Similarly, our method exhibits a significant improvement in *Recall* compared to the keyword-based single-point service matching method. The *Recall* of composition scheme discovery in both loose and strict modes is lower than that of single-point service discovery. Additionally, the *Recall* of the loose mode composition scheme is higher compared to the strict mode.

Based on *Precision* and *Recall*, we can further obtain the *F-Measure* under the four conditions mentioned above, as shown in Fig. 14. It can be observed that both single-point service discovery and composition scheme discovery exhibit good performance, reflecting that our proposed method's matched results indeed align with subjective judgments of service requirements. This confirms the effectiveness of our method.

Although our method has demonstrated promising results, during the process of experimental evaluation, we have become aware of its limitations. Firstly, the performance of the method is largely dependent on the quality of the ontology. The more detailed and hierarchically structured the ontology descriptions are, the better the method performs. This poses a challenge when seeking or constructing ontologies. Secondly, due to the diverse structural characteristics of different ontologies, we opted for a generic approach using semantic distance to calculate semantic similarity between two ontology concepts. However, it would be worthwhile to tailor specific semantic similarity calculation methods for different ontologies based on their unique characteristics. This could involve incorporating relationship similarity between concepts as a partial representation of concept similarity. Lastly, in the process of constructing the composition dependency graph, we utilized the overall matching score of the output-input set as a representation of the composition potential between two services. This resulted in situations where a specific pair of concepts between two services exhibited strong composition potential, but due to issues with the overall quantity, the overall composition potential was low. Setting a threshold further led to the loss of some potential composition schemes tailored to specific needs, emphasizing the need for more attention to concepts with very strong matching between output and input.

Finally, in our future work, we will focus on addressing the limitations of our method. Additionally, we are also paying attention to some emerging API-related technologies and their potential integration with our method, indicating our next research direction. For instance, GraphQL API(See https://graphql.org/), heralded as the next generation of API, introduces many novel features and advantages as a replacement for RESTful API. We contemplate incorporating ontology knowledge into the GraphQL API information description and interaction process through JSON-LD (see https://json-ld.org/). Subsequently, in conjunction with our proposed method of leveraging ontological semantics, we believe that this can bring about more positive impacts, both in the interaction between users and GraphQL API and in the interoperability between GraphQL APIs. In conclusion, we genuinely hope

that our proposed method can bring more "intelligence" to the application of service APIs.

Funding This study was financially supported by the Science and Technology Development Plan of Jilin Province (Grant No. 20220203175SF).

Declarations

Conflict of interest The authors declare that they have no Conflict of interest.

References

1. Erickson J, Siau K (2008) Web services, service-oriented computing, and service-oriented architecture: separating hype from reality. J Datab Manag (JDM) 19(3):42–54
2. Qi L, He Q, Chen F, Zhang X, Dou W, Ni Q (2020) Data-driven web apis recommendation for building web applications. IEEE Trans Big Data 8(3):685–698
3. Yu Q, Liu X, Bouguettaya A, Medjahed B (2008) Deploying and managing web services: issues, solutions, and directions. VLDB J 17:537–572
4. Crasso M, Zunino A, Campo M (2011) A survey of approaches to web service discovery in service-oriented architectures. J Datab Manag (JDM) 22(1):102–132
5. Garriga M, Flores A, Cechich A, Zunino A (2015) Web services composition mechanisms: a review. IETE Tech Rev 32(5):376–383
6. Curbera F, Duftler M, Khalaf R, Nagy W, Mukhi N, Weerawarana S (2002) Unraveling the web services web: an introduction to soap, wsdl, and uddi. IEEE Internet Comput 6(2):86–93
7. Garriga M, Mateos C, Flores A, Cechich A, Zunino A (2016) Restful service composition at a glance: A survey. J Netw Comput Appl 60:32–53
8. Adamczyk P, Smith PH, Johnson RE, Hafiz M (2011) Rest and web services: In theory and in practice. REST: from research to practice, 35–57
9. Cheng B, Zhao S, Li C, Chen J (2016) A web services discovery approach based on mining underlying interface semantics. IEEE Trans Knowl Data Eng 29(5):950–962
10. Zhang F, Zeng Q, Duan H, Liu C (2019) Composition context-based web services similarity measure. IEEE Access 7:65195–65206
11. Halilali MS, Gouardères E, Gaio M, Devin F (2022) Geospatial web services discovery through semantic annotation of wps. ISPRS Int J Geo Inf 11(4):254
12. Facciorusso C, Field S, Hauser R, Hoffner Y, Humbel R, Pawlitzek R, Rjaibi W, Siminitz C (2003) A web services matchmaking engine for web services. In: E-Commerce and Web Technologies: 4th International Conference, EC-Web, Prague, Czech Republic, September 2-5, 2003. Proceedings 4, pp. 37–49. Springer
13. Sajjanhar A, Hou J, Zhang Y (2004) Algorithm for web services matching. In: Advanced web technologies and applications: 6th Asia-Pacific Web Conference, APWeb 2004, Hangzhou, China, April 14-17, 2004. Proceedings 6, pp. 665–670. Springer
14. Paliwal AV, Shafiq B, Vaidya J, Xiong H, Adam N (2011) Semantics-based automated service discovery. IEEE Trans Serv Comput 5(2):260–275
15. Ngan LD, Kanagasabai R (2013) Semantic web service discovery: state-of-the-art and research challenges. Pers Ubiquit Comput 17:1741–1752
16. Dantas JRV, Farias PPM (2020) An architecture for restful web service discovery using semantic interfaces. International Journal on Semantic Web and Information Systems (IJSWIS) 16(1):1–24
17. Wang S, Zhu X, Yang F (2014) Efficient qos management for qos-aware web service composition. Int J Web Grid Serv 10(1):1–23
18. Jatoth C, Gangadharan G, Buyya R (2015) Computational intelligence based qos-aware web service composition: a systematic literature review. IEEE Trans Serv Comput 10(3):475–492

19. Xing Y, Li Y, Yu X (2005) Using semantic matching, research on semantic web services composition. In: Advances in web intelligence: third international atlantic web intelligence conference, AWIC 2005, Lodz, Poland, June 6-9, 2005. Proceedings 3, pp. 445–450. Springer
20. Kil H, Nam W (2013) Semantic web service composition via model checking techniques. Int J Web Grid Serv 9(4):339–350
21. Abid A, Rouached M, Messai N (2020) Semantic web service composition using semantic similarity measures and formal concept analysis. Multim Tool Appl 79:6569–6597
22. Pautasso C (2009) Restful web service composition with bpel for rest. Data Knowl Eng 68(9):851–866
23. Arch-Int N, Arch-Int S, Sonsilphong S, Wanchai P (2017) Graph-based semantic web service composition for healthcare data integration. J Healthc Eng. https://doi.org/10.1155/2017/4271273
24. Zhovtobryukh D (2007) A petri net-based approach for automated goal-driven web service composition. Simulation 83(1):33–63
25. Ed-Douibi H, Cánovas Izquierdo JL, Cabot J (2017) Example-driven web api specification discovery. In: Modelling foundations and applications: 13th European Conference, ECMFA 2017, Held as Part of STAF 2017, Marburg, Germany, July 19-20, 2017, Proceedings 13, pp. 267–284. Springer
26. Karavisileiou A, Mainas N, Petrakis EG (2020) Ontology for openapi rest services descriptions. In: 2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI), pp. 35–40. IEEE
27. Jayawardhana UK, Gorsevski PV (2019) An ontology-based framework for extracting spatio-temporal influenza data using twitter. Int J Digital Earth 12(1):2–24
28. Mohamed M, Oussalah M (2020) A hybrid approach for paraphrase identification based on knowledge-enriched semantic heuristics. Lang Resour Eval 54:457–485

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Haijun Gu[1] · Yingyu Ma[1] · Siqi Wang[1] · Xincheng Chen[1] · Weihua Su[1]

✉ Haijun Gu
guhj@jlu.edu.cn

Yingyu Ma
mayy21@mails.jlu.edu.cn

Siqi Wang
siqi22@mails.jlu.edu.cn

Xincheng Chen
chenxc21@mails.jlu.edu.cn

Weihua Su
suwh22@mails.jlu.edu.cn

[1] College of Communication Engineering, Jilin University, 5372 Nanhu Road, Changchun 130012, China

Computing is a copyright of Springer, 2024. All Rights Reserved.